# FOR LOOP STATEMENT

**NAMA:** Ihsanta Zaki Sanjaya

**KELAS:** 1 D4 Teknik Komputer B

**NRP:** 3223600058

**DOSEN:** Reni Soelistijorini, B.Eng., MT

**MATA KULIAH:** Praktikum Rangkaian Logika 2

**TANGGAL:** Kamis, 18 April 2024

**D4 TEKNIK KOMPUTER**
**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**
**2024**

CONDITIONAL dan SELECTED SIGNAL ASSIGNMENT

## 1. TUJUAN

- Mahasiswa dapat membuat progam dasar VHDL menggunakan *for statemen.*
- Mahasiswa dapat membuat rangkaian logika sederhana menggunakan *for st atement.*

## 2. ALAT DAN BAHAN

Alat dan Bahan untuk praktikum ini yaitu:

1. PC beserta software ISE Design Suite 13.1
2. Xilinx Spartan 3
3. Downloader JTAG USB
4. Power Supply 5V

## 3. PROSEDUR PERCOBAAN

A. Decoder 3 to 8

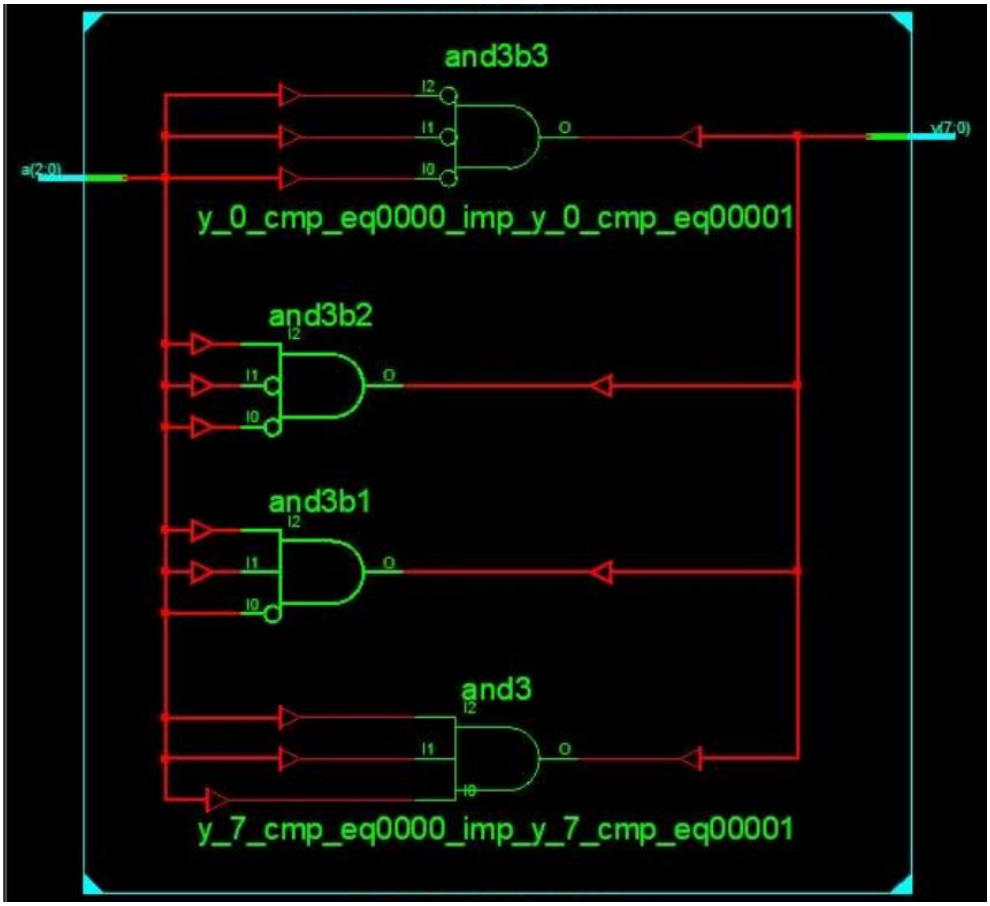
1. Program untuk menampilkan decoder 3 to 8 dengan for statement adalah sebagai berikut :

**Konfigurasi pada file .vhd Implementation**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity dec38 is
   port (
      a : in STD_LOGIC_VECTOR(2 downto 0);
      y : out STD_LOGIC_VECTOR(7 downto 0) -- Corrected the range here
   );
end dec38;
architecture Behavioral of dec38 is
begin
   process(a)
      variable j : integer;
   begin
      j := to_integer(unsigned(a));
      for i in 0 to 7 loop
         if i = j then
            y(i) <= '1';
         else
            y(i) <= '0';
         end if;
      end loop;
   end process;
end Behavioral;
```

2. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.
3. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start with a schematic of the top-level block, kemudian klik OK.

4. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.

5. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.
6. 


7. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
8. Konfigurasikan file test bench (main_tb.vhd)

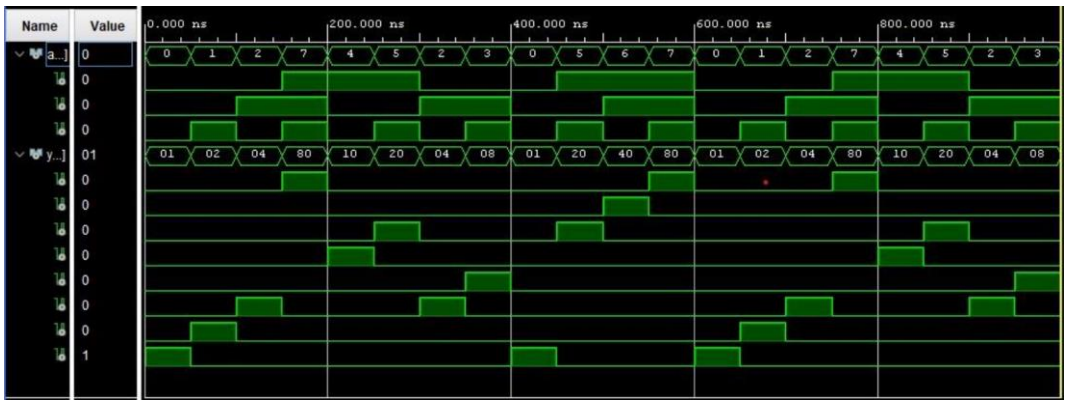
**Konfigurasi pada file main_tb.vhd Simulation**

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY main_tb IS
END main_tb;

ARCHITECTURE behavior OF main_tb IS
   COMPONENT dec38
   PORT(
       a : IN  std_logic_vector(2 downto 0);
       y : OUT  std_logic_vector(7 downto 0)
      );
   END COMPONENT;
   -- Inputs
   signal a : std_logic_vector(2 downto 0) := (others => '0');
   -- Outputs
   signal y : std_logic_vector(7 downto 0);

BEGIN
   uut: dec38 PORT MAP (
        a => a,
        y => y
      );
   stim_proc: process
   begin
      wait for 150 ns;
      a(2) <= not a(2);
   end process;
            stim_proc1: process
   begin
      wait for 100 ns;
      a(1) <= not a(1);
   end process;
            stim_proc2: process
   begin
      wait for 50 ns;
      a(0) <= not a(0);
   end process;

END;
```

9. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click “Behavioral Check Syntax-Run”. Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run untuk menampilkan simulasi.

10. Gunakan fitur “Zoom to full view” untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

Hasil time diagram untuk decoder 3 to 8 dengan for statement:




*1 Time Diagram Decoder 2 to 8 dengan For Statement*

B. Binary to BCD Converter

1. Buatlah file baru untuk desain Binary to BCD Converter. Programnya adalah sebagai berikut :

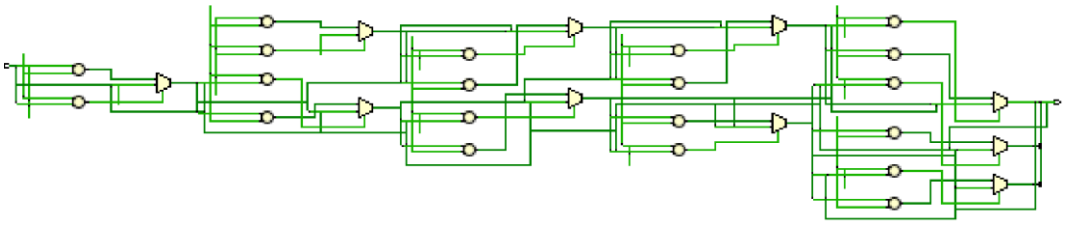
**Konfigurasi pada file .vhd Implementation**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_logic_unsigned.all;

entity BCD is
port(
        b:in STD_LOGIC_VECTOR (7 downto 0);
        p:out STD_LOGIC_VECTOR (9 downto 0)
  );
 End BCD;
architecture Behavioral of BCD is
begin
        bcd1:process(b)
        variable z :STD_LOGIC_VECTOR(17 downto 0);
        begin
        for i in 0 to 17 loop
        z(i):= '0';
        end loop;

        z(10 downto 3):=b;
        for i in 0 to 4 loop
if z(11 downto 8)> 4 then
z(11 downto 8):= z(11 downto 8)+3;
end if;
if z(15 downto 12)>4 then
z(15 downto 12):= z(15 downto 12)+3;
end if;
z(17 downto 1):= z(16 downto 0);
end loop;
p <= z(17 downto 8);
end process bcd1;
end Behavioral ;
```

2. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.
3. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start with a schematic of the top-level block, kemudian klik OK.

4. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.

5. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.



6. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
7. Konfigurasikan file test bench (main_tb.vhd)

**Konfigurasi pada file main_tb.vhd Simulation**

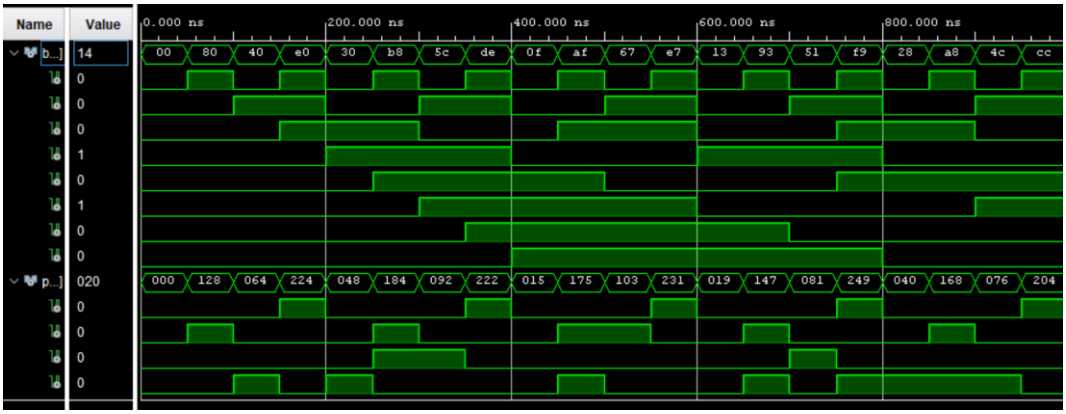
```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY main_tb IS
END main_tb;

ARCHITECTURE behavior OF main_tb IS
   COMPONENT BCD
   PORT(
       b : IN  std_logic_vector(7 downto 0);
       p : OUT  std_logic_vector(9 downto 0)
      );
   END COMPONENT;

  --Inputs
  signal b : std_logic_vector(7 downto 0) := (others => '0');

           --Outputs
  signal p : std_logic_vector(9 downto 0);

BEGIN

           -- Instantiate the Unit Under Test (UUT)
  uut: BCD PORT MAP (
       b => b,
       p => p
      );


  stim_proc: process
  begin
    wait for 400 ns;
                      b(0) <= not b(0);
  end process;
             stim_proc1: process
  begin
    wait for 350 ns;
                      b(1) <= not b(1);
  end process;
  stim_proc2: process
```

```
    begin
      wait for 300 ns;
                    b(2) <= not b(2);
    end process;
    stim_proc3: process
    begin
      wait for 250 ns;
                    b(3) <= not b(3);
    end process;
    stim_proc4: process
    begin
      wait for 200 ns;
                    b(4) <= not b(4);
    end process;
    stim_proc5: process
    begin
      wait for 150 ns;
                    b(5) <= not b(5);
    end process;
    stim_proc6: process
    begin
      wait for 100 ns;
                    b(6) <= not b(6);
    end process;
    stim_proc7: process
    begin
      wait for 50 ns;
                    b(7) <= not b(7);
    end process;

  END;
```

8. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click "Behavioral Check Syntax-Run". Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run untuk menampilkan simulasi.

9. Gunakan fitur "Zoom to full view" untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

Hasil time diagram untuk Binary to BCD Converter:




*2 Binary to BCD Converter*

C. Gray Code Converter – Biner to Gray

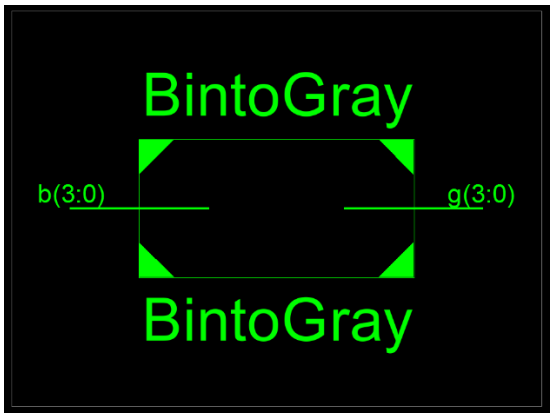
Program untuk menampilkan Gray Code Converter – Biner to Gray adalah sebagai berikut :

**Konfigurasi pada file .vhd Implementation**

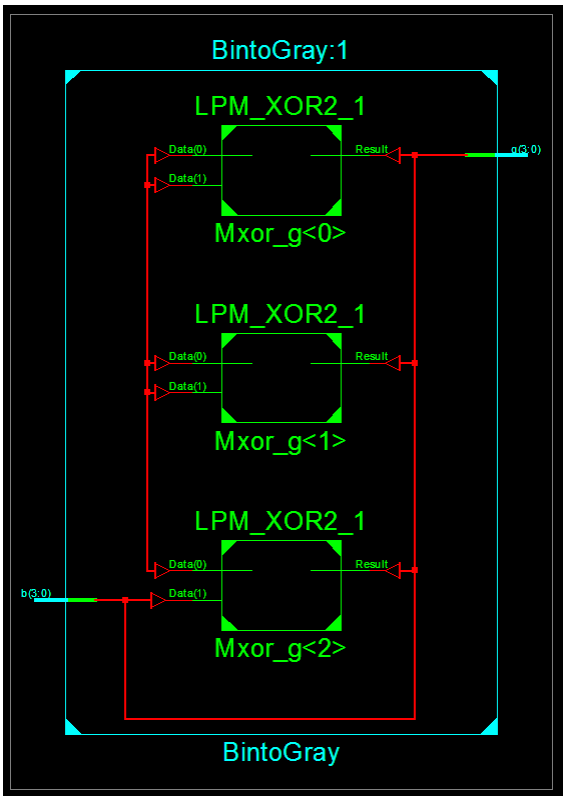
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity BintoGray is
port(
b:in STD_LOGIC_VECTOR(3 downto 0);
g:out STD_LOGIC_VECTOR(3 downto 0)
);
end BintoGray;

architecture Behavioral of BintoGray is

begin

process(b)
begin
g(3)<= b(3);
for i in 2 downto 0 loop
g(i )<= b(i +1)xor b(i );
end loop;
end process;
end Behavioral;
```

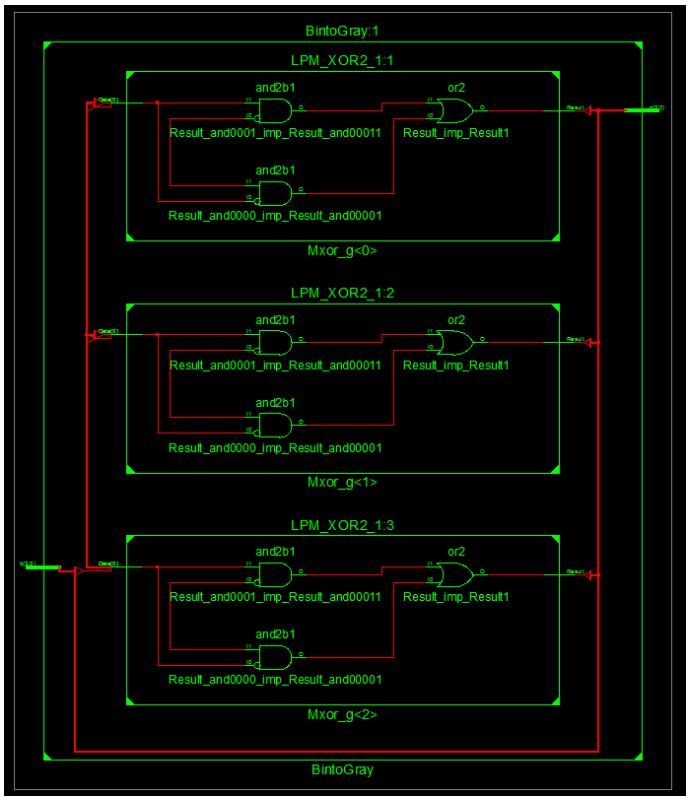
1. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.
2. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start with a schematic of the top-level block, kemudian klik OK.

3. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.




4. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.



5. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
6. Konfigurasikan file test bench (main_tb.vhd)

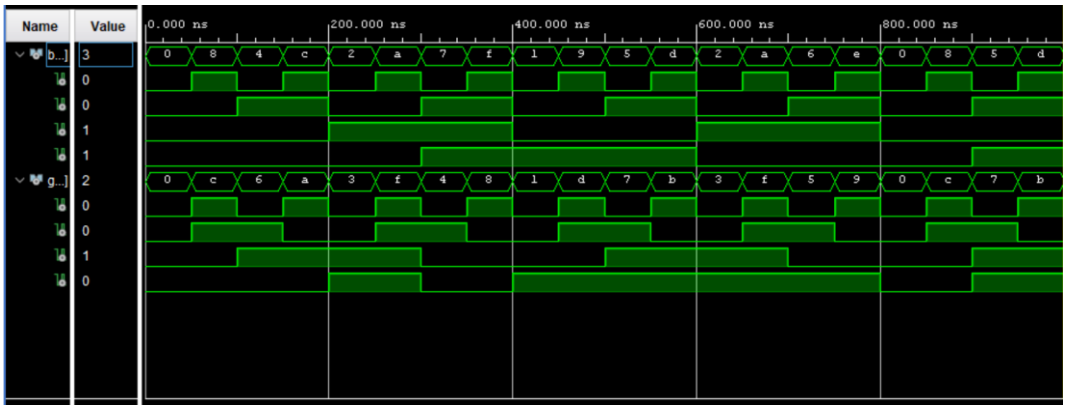
**Konfigurasi pada file main_tb.vhd Simulation**

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;
ENTITY main_tb IS
END main_tb;
ARCHITECTURE behavior OF main_tb IS
   -- Component Declaration for the Unit Under Test (UUT)
   COMPONENT BTG
   PORT(
       b : IN  std_logic_vector(3 downto 0);
       g : OUT  std_logic_vector(3 downto 0)
      );
   END COMPONENT;
  --Inputs
  signal b : std_logic_vector(3 downto 0) := (others => '0');
          --Outputs
  signal g : std_logic_vector(3 downto 0);
  -- No clocks detected in port list. Replace <clock> below with
  -- appropriate port name

BEGIN
          -- Instantiate the Unit Under Test (UUT)
  uut: BTG PORT MAP (
       b => b,
       g => g
      );
  -- Clock process definitions
  -- Stimulus process
  stim_proc: process
  begin
    wait for 300 ns;
                    b(0) <= not b(0);
  end process;
           stim_proc1: process
  begin
    wait for 200 ns;
                    b(1) <= not b(1);
  end process;
 stim_proc2: process
  begin
    wait for 100 ns;
                    b(2) <= not b(2);
  end process;
          stim_proc3: process
  begin
    wait for 50 ns;
                    b(3) <= not b(3);
  end process;
END;
```

7. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click “Behavioral Check Syntax-Run”. Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run untuk menampilkan simulasi.

8. Gunakan fitur “Zoom to full view” untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

Hasil time diagram untuk Gray Code Converter – Biner to Gray:




*3 Time Diagram Gray Code Converter – Biner to Gray*

D. Gray Code Converter – Gray to Biner

Program untuk menampilkan Gray Code Converter – Gray to Biner adalah sebagai berikut :

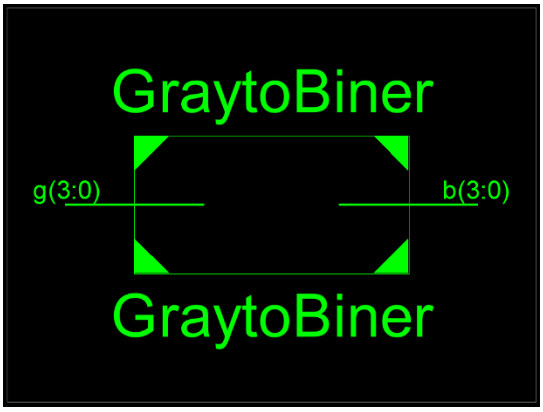
**Konfigurasi pada file .vhd Implementation**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity GraytoBiner is
port(
g:in STD_LOGIC_VECTOR(3 downto 0); b:out STD_LOGIC_VECTOR(3 downto
0)
);
end GraytoBiner;

architecture Behavioral of GraytoBiner is

begin

process(g)     variable bv :STD_LOGIC_VECTOR(3 downto 0); begin bv(3):=
g(3); for i in 2 downto 0 loop bv(i ):= bv(i +1)xor g(i ); end loop;
b <= bv; end process;
end Behavioral;
```

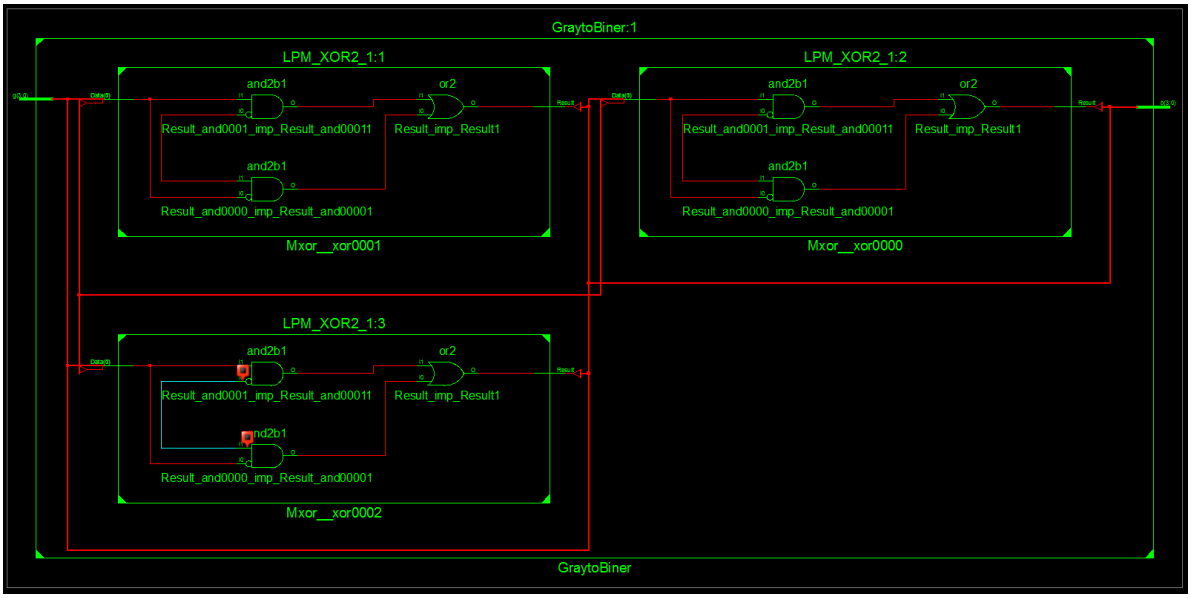
1. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.
2. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start

with a schematic of the top-level block, kemudian klik OK.

3. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.




4. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.




5. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
6. Konfigurasikan file test bench (main_tb.vhd)

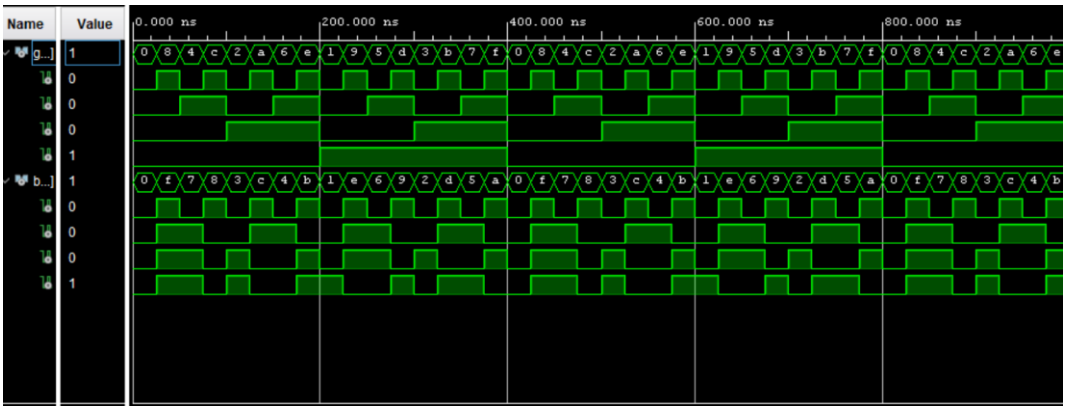
**Konfigurasi pada file main_tb.vhd Simulation**

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;
ENTITY main_tb IS
END main_tb;
ARCHITECTURE behavior OF main_tb IS
   -- Component Declaration for the Unit Under Test (UUT)
   COMPONENT GTB
   PORT(
       g : IN  std_logic_vector(3 downto 0);
       b : OUT  std_logic_vector(3 downto 0)
      );
   END COMPONENT;
  --Inputs
  signal g : std_logic_vector(3 downto 0) := (others => '0');
          --Outputs
  signal b : std_logic_vector(3 downto 0);
  -- No clocks detected in port list. Replace <clock> below with
  -- appropriate port name
BEGIN
          -- Instantiate the Unit Under Test (UUT)
  uut: GTB PORT MAP (
       g => g,
       b => b
      );
  -- Stimulus process
  stim_proc: process
  begin
    wait for 300 ns;
                    g(0) <= not g(0);
  end process;
          stim_proc1: process
  begin
    wait for 200 ns;
                    g(1) <= not g(1);
  end process;
          stim_proc2: process
  begin
    wait for 100 ns;
                    g(2) <= not g(2);
  end process;
          stim_proc3: process
  begin
    wait for 50 ns;
                    g(3) <= not g(3);
  end process;
END;
```

7. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click “Behavioral Check Syntax-Run”. Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run untuk menampilkan simulasi.

8. Gunakan fitur “Zoom to full view” untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

Hasil time diagram untuk Gray Code Converter – Gray to Biner:




*4 Time Diagram Gray Code Converter – Gray to Biner*

E. Multiplier

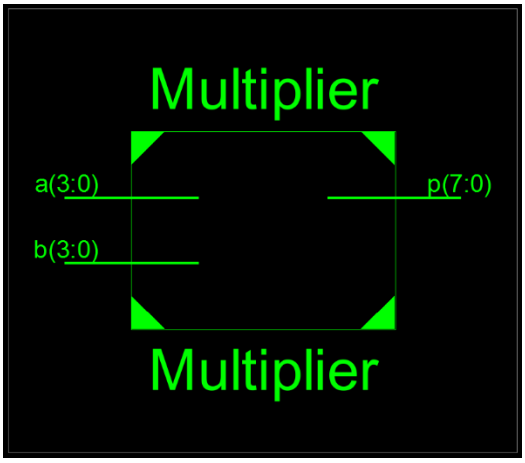
Program untuk menampilkan Multiplier adalah sebagai berikut :

**Konfigurasi pada file .vhd Implementation**

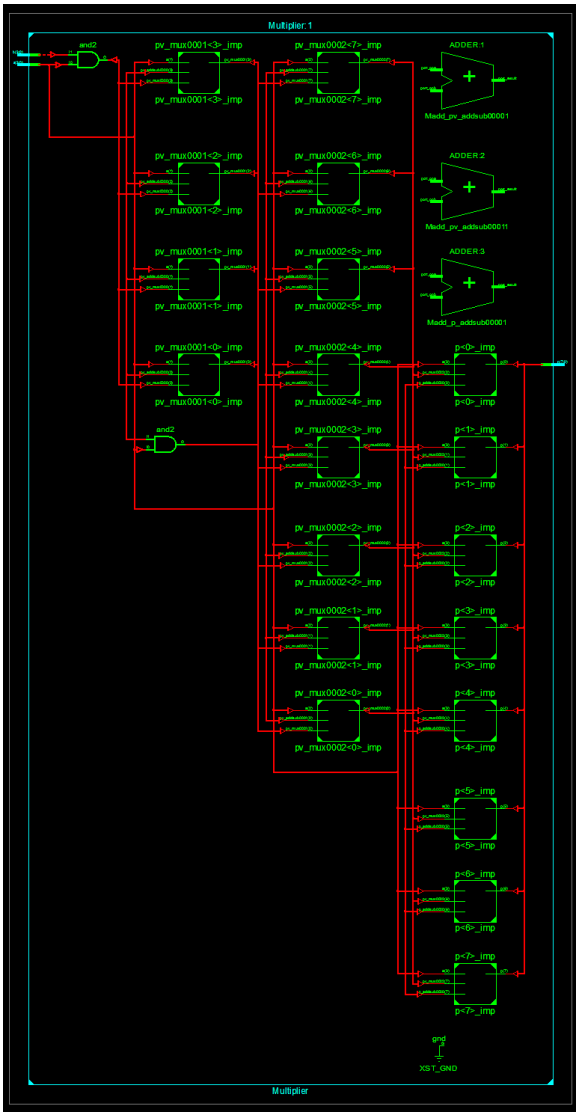
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.all;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;
-- Uncomment the following library declaration if instantiating
-- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Multiplier is
port(
a:in STD_LOGIC_VECTOR(3 downto 0); b:in STD_LOGIC_VECTOR(3 downto
0); p:out STD_LOGIC_VECTOR(7 downto 0)
);
end Multiplier;
architecture Behavioral of Multiplier is
begin

process(a,b) variable pv,bp :STD_LOGIC_VECTOR(7 downto 0); begin pv
:="00000000"; bp :="0000"& b; for i in 0 to 3 loop
if a(i)= '1' then
pv := pv + bp; end if; bp := bp(6 downto 0) & '0' ; end loop; p <= pv; end
process;

end Behavioral;
```

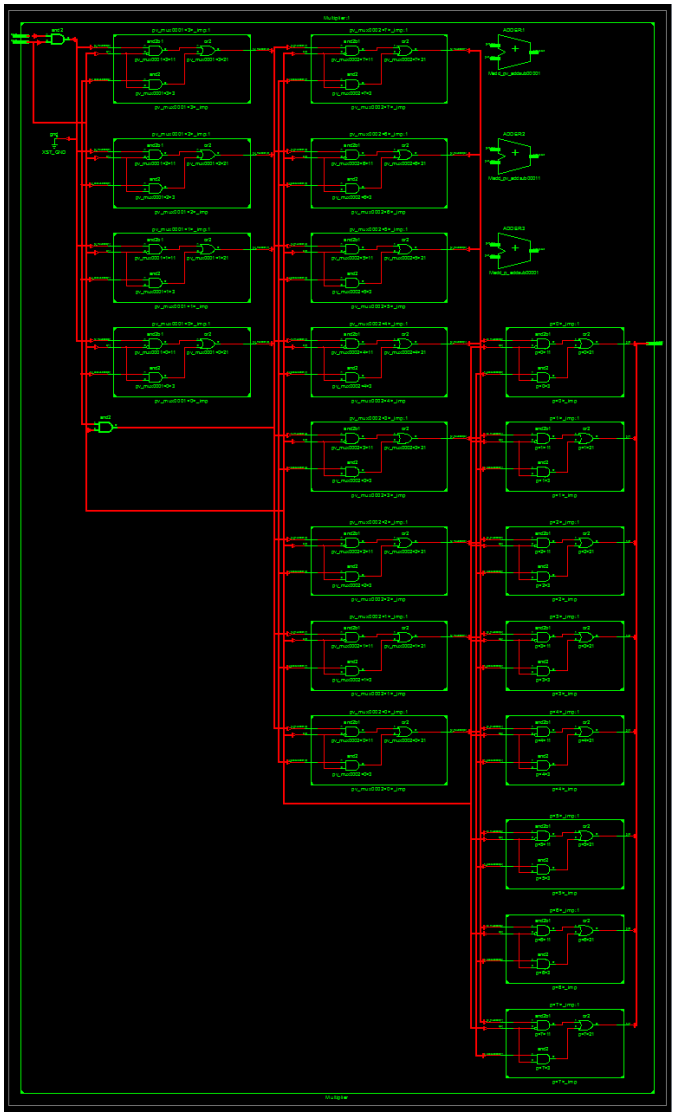
1. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.
2. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start with a schematic of the top-level block, kemudian klik OK.

3. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.




4. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.





5. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
6. Konfigurasikan file test bench (main_tb.vhd)

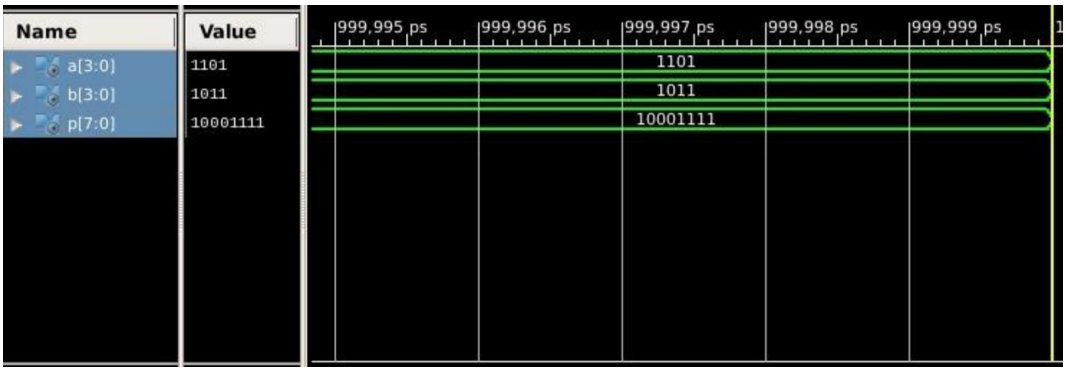
**Konfigurasi pada file main_tb.vhd Simulation**

```
LIBRARY ieee;
 USE ieee.std_logic_1164.ALL;
 ENTITY MAIN_TB IS
 END MAIN_TB;
 ARCHITECTURE behavior OF MAIN_TB IS
    COMPONENT Multiplier
    PORT(
        a : IN  std_logic_vector(3 downto 0);
        b : IN  std_logic_vector(3 downto 0);
        p : OUT  std_logic_vector(7 downto 0)
       );
    END COMPONENT;
   signal a : std_logic_vector(3 downto 0) := (others => '0');
   signal b : std_logic_vector(3 downto 0) := (others => '0');
   signal p : std_logic_vector(7 downto 0);
 BEGIN
   uut: Multiplier PORT MAP (
        a => a,
        b => b,
        p => p
       );
  stim_proc: process
   begin
     wait for 50ns;
                    a(0) <= not a(0);
 a <= "1101"; -- 13
 b <= "1011"; -- 11
   end process;
 END;
```

7. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click “Behavioral Check Syntax-Run”. Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run untuk menampilkan simulasi.

8. Gunakan fitur “Zoom to full view” untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

Hasil time diagram untuk Multiplier:




*5 Time Diagram Multiplier*

F. Divider

Program untuk menampilkan Divider adalah sebagai berikut :

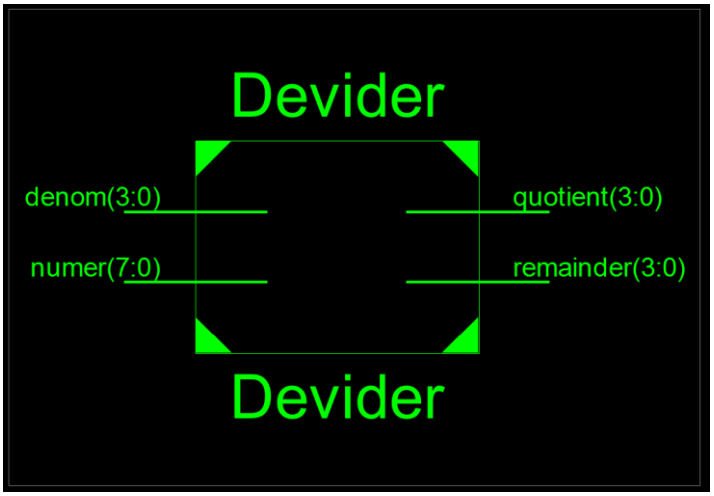
**Konfigurasi pada file .vhd Implementation**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
Use IEEE.STD_LOGIC_unsigned.all;
entity Devider is
port(
numer :in STD_LOGIC_VECTOR(7 downto 0);
denom :in STD_LOGIC_VECTOR(3 downto 0);
quotient :out STD_LOGIC_VECTOR(3 downto 0);
remainder :out STD_LOGIC_VECTOR(3 downto 0));
end Devider;
architecture Behavioral of Devider is
begin
process(numer,denom )
variable d, n1 :STD_LOGIC_VECTOR(4 downto 0);
variable n2 :STD_LOGIC_VECTOR(3 downto 0);
begin
d  := '0' & denom ;
n2 := numer(3 downto 0);
n1 := '0' & numer (7 downto 4);
for i in 0 to 3 loop n1 :=
n1(3 downto 0)& n2(3); n2 :=
n2(2 downto 0)& '0' ;
if n1 >= d then
n1 := n1-d;
n2(0):= '1' ;
end if;
end loop;
quotient <= n2;
remainder <= n1(3 downto 0);
end process;
end Behavioral;
```

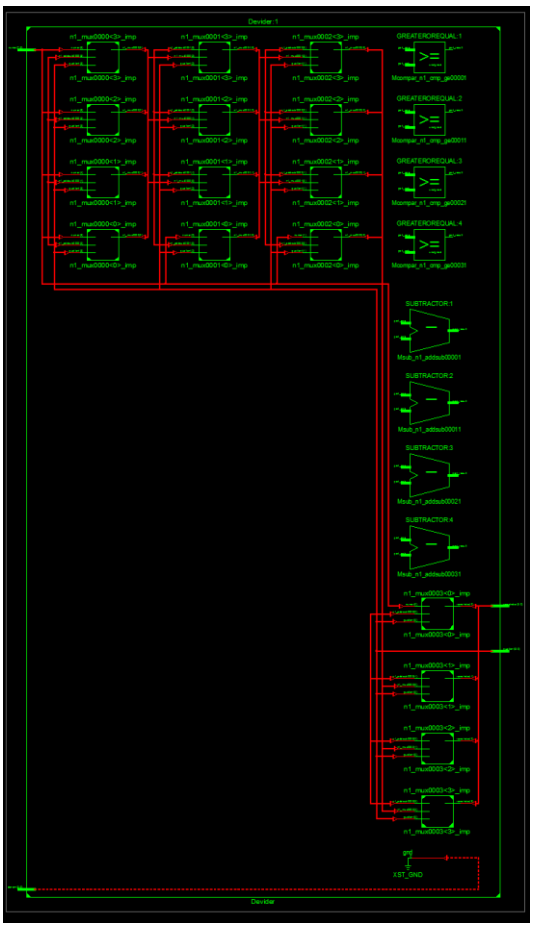
1. Simpan proyek, lalu double klik “Synthesize-XST” untuk mensintesis program (mengompilasi program). Jika program tidak terdapat kesalahan maka akan muncul notiffikasi “File Succeed”, jika terdapat kesalahan akan muncul “File Failed” atau “Error”.

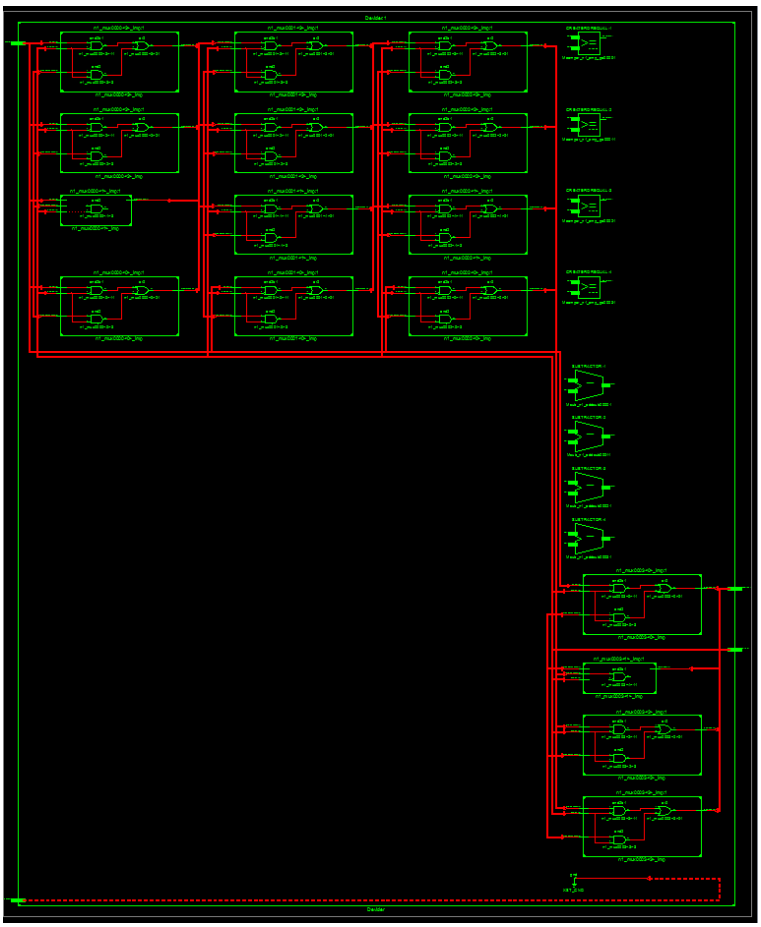
2. Setelah berhasil dikompilasi, selanjutnya klik kanan pada tanda + disebelah kiri Synthesize -XST. Double click pada View RTL Simulation, lalu pilih Start with a schematic of the top-level block, kemudian klik OK.

3. Terdapat tampilan RTL Schematic berisikan rangkaian yang sudah kita buat.




4. Double click pada diagram block, maka akan muncul detail rangkaian logikanya.





5. Kembali ke menu utama, pilih Simulation pada View yang berada di kolom Design. Klik kanan pada nama file project. Klik “New Source” dan pilih “VHDL Test bench”, serta isikan nama filenya dengan nama main_tb. Pada bagian yang menunjukan file yang akan disimulasikan. Pilih Next dan Finish.
6. Konfigurasikan file test bench (main_tb.vhd)

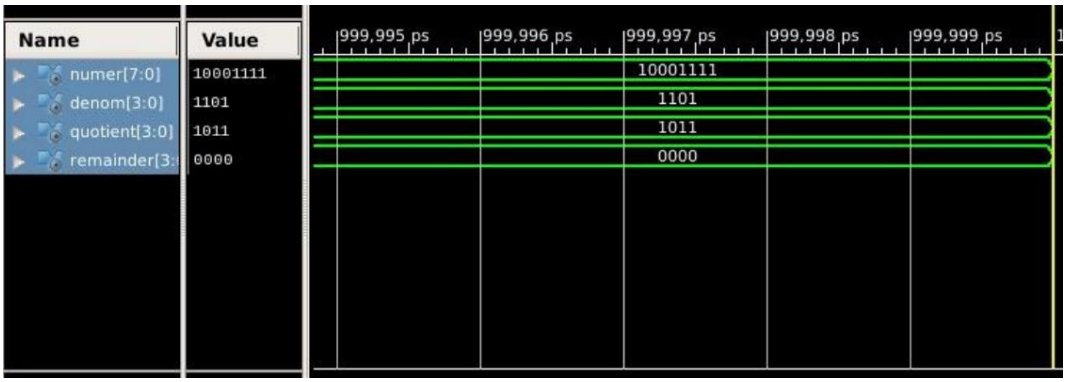
**Konfigurasi pada file main_tb.vhd Simulation**

```
library IEEE;
 use IEEE.STD_LOGIC_1164.ALL;

 entity devider_tb is
 end devider_tb;

 architecture Behavioral of devider_tb is
    component devider
       Port ( numer : in STD_LOGIC_VECTOR(7 downto 0);
             denom : in STD_LOGIC_VECTOR(3 downto 0);
             quotient : out STD_LOGIC_VECTOR(3 downto 0);
             remainder : out STD_LOGIC_VECTOR(3 downto 0)
            );
    end component;
   signal numer : STD_LOGIC_VECTOR(7 downto 0):=(others => ‘0’);
    signal denom : STD_LOGIC_VECTOR(3 downto 0) :=(others => ‘0’);
    signal quotient : STD_LOGIC_VECTOR(3 downto 0);
    signal remainder : STD_LOGIC_VECTOR(3 downto 0);

 begin
    uut: devider port map (
 numer => numer,
 denom => denom,
 quotient => quotient,
 remainder => remainder
 );
 stimulus_proc: process
begin
     wait for 50 ns;
        numer <= "10001111"; -- 143
           denom <= "1101"; -- 13
 end process;
 END;
```

7. Setelah berhasil melakukan perubahan, selanjutnya klik kanan pada tanda + disebelah kiri dari Isim Simulator. Double click “Behavioral Check Syntax-Run”. Ketika berhasi, klik kanan pada Simulate Behavioral Model , pilih Run untuk menampilkan simulasi.

8. Gunakan fitur “Zoom to full view” untuk melihat simulasi secara detail. Sehingga akan terlihat timing diagram seperti berikut.

Hasil time diagram untuk Divider:




*6 Time Diagram Divider*

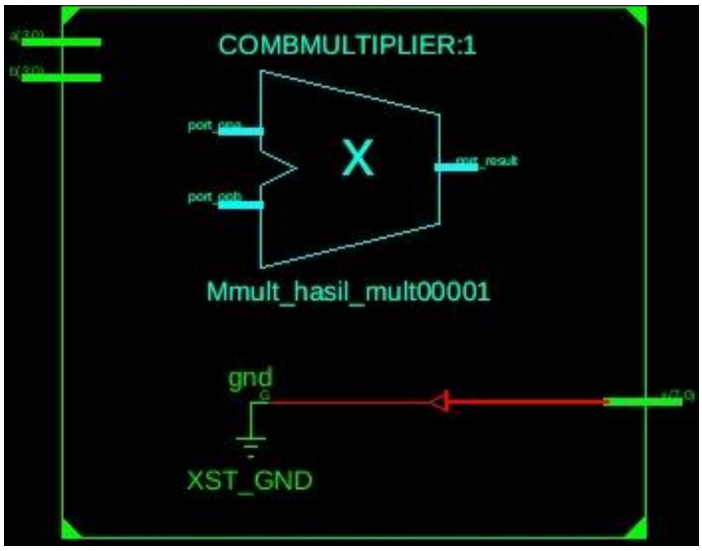
G. Tugas

Program dengan persamaan x = (a * b)/2

**Konfigurasi pada file .vhd Implementation**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.numeric_std.ALL;

entity tugas is
        port( a : in std_logic_vector (3 downto 0);
                b : in std_logic_vector (3 downto 0);
                x : out std_logic_vector (7 downto 0));
end tugas;
architecture Behavioral of tugas is
        function persamaan(a,b : unsigned)
                return unsigned is
variable hasil : unsigned (7 downto 0);
begin
        hasil := (a * b) / 2;
        return hasil;
end function;
signal nilai1, nilai2 : unsigned (3 downto 0);
begin
        nilai1 <= unsigned(a);
        nilai2 <= unsigned(b);
        x <= std_logic_vector(persamaan(nilai1, nilai2));
end Behavioral;
```

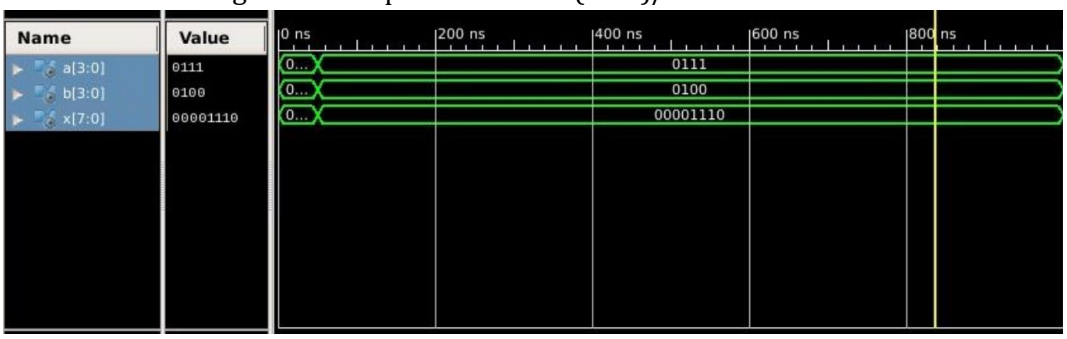
RTL



**Konfigurasi pada file main_tb.vhd Simulation**

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY tugas_tb IS END tugas_tb;
ARCHITECTURE behavior OF tugas_tb IS

        a : IN std_logic_vector(3 downto 0);
        b : IN std_logic_vector(3 downto 0);
        x : OUT std_logic_vector(7 downto 0) );
END COMPONENT;
        signal a : std_logic_vector(3 downto 0) := (others => '0');
        signal b : std_logic_vector(3 downto 0) := (others => '0');
        signal x : std_logic_vector(7 downto 0);
BEGIN
uut: tugas PORT MAP (
        a => a,
        b => b,
        x => x );
stimulus_proc: process
begin
        wait for 50 ns;
        a <= "0111"; -- 7
        b <= "0100"; -- 4
end process;
END;
```

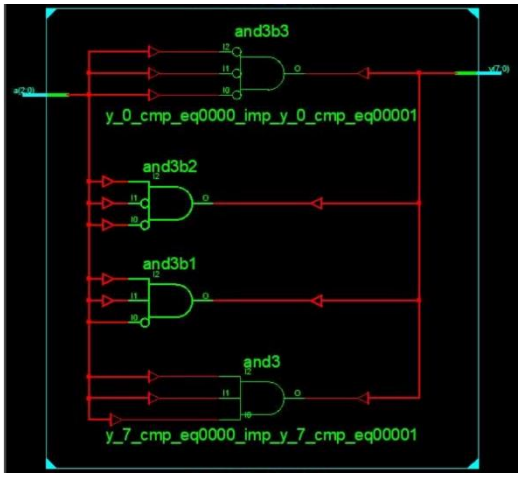
Hasil time diagram untuk persamaan x = (a * b)/2



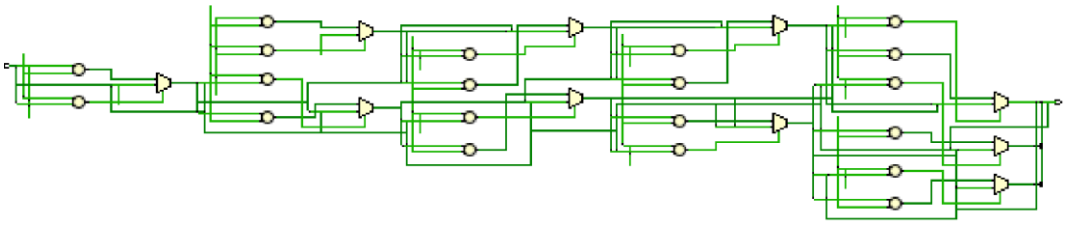

*7 Time Diagram persamaan x=(a*b)/2*

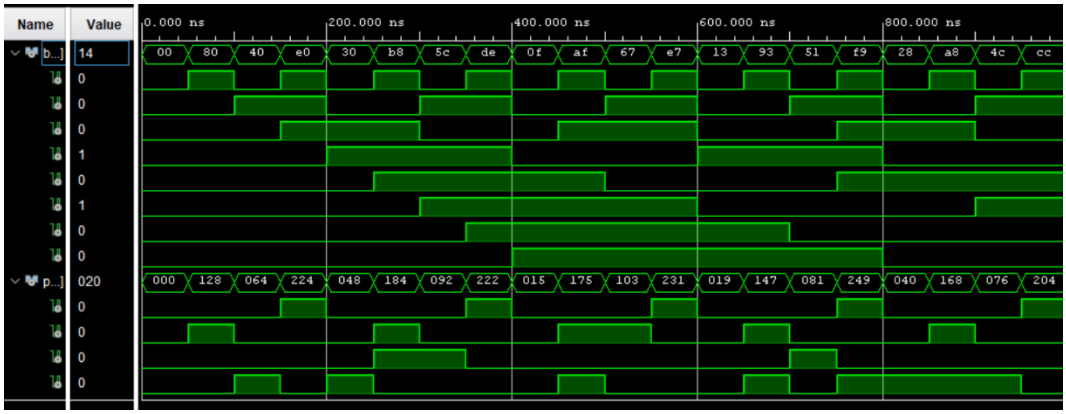
## H. HASIL DATA PERCOBAAN

a. Decoder 3 to 8

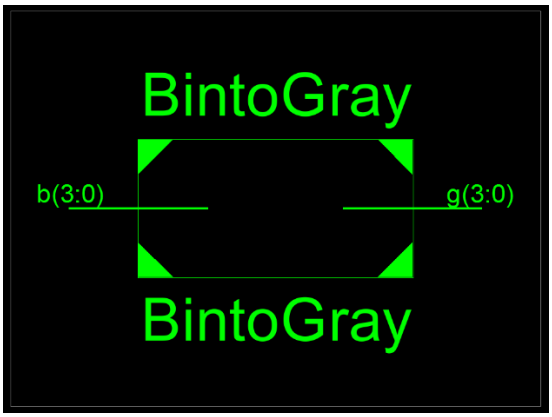


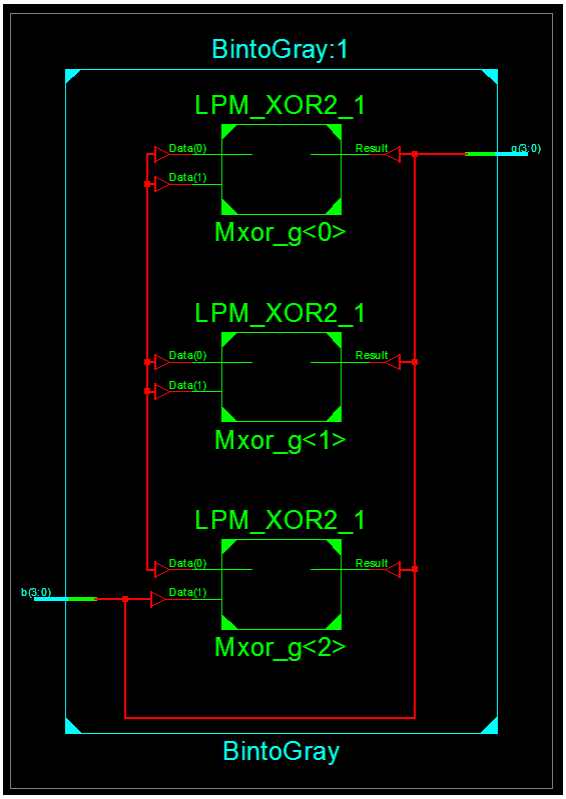
b. Binary to BCD

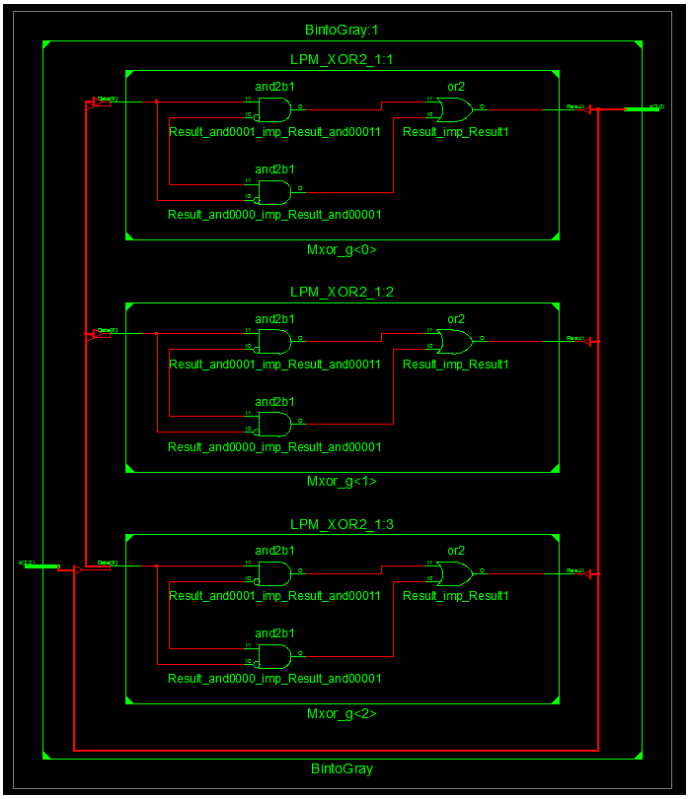


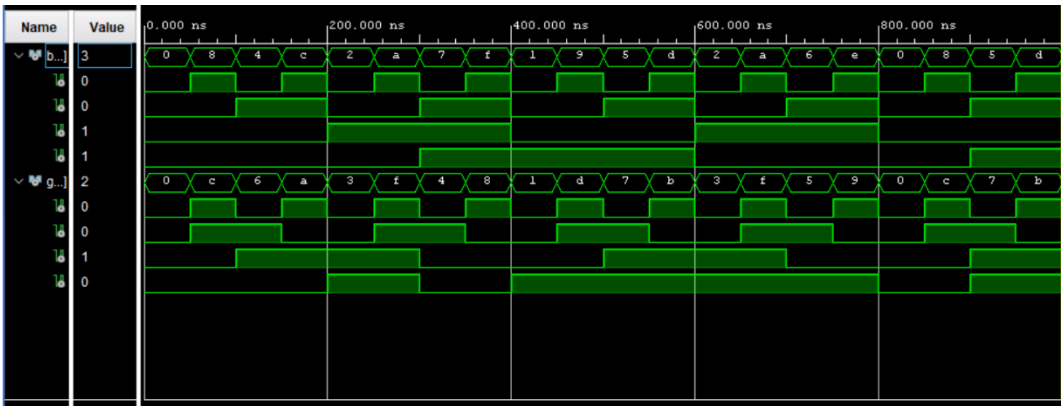


c. Binary to Gray

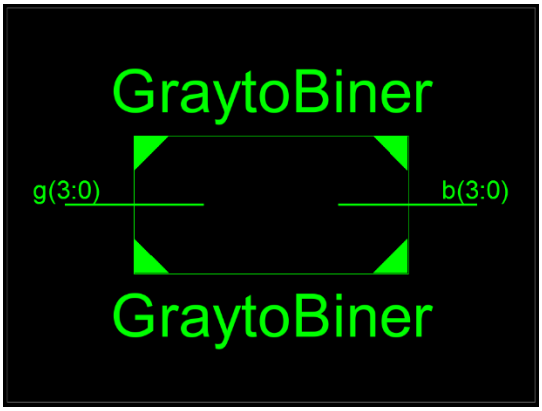


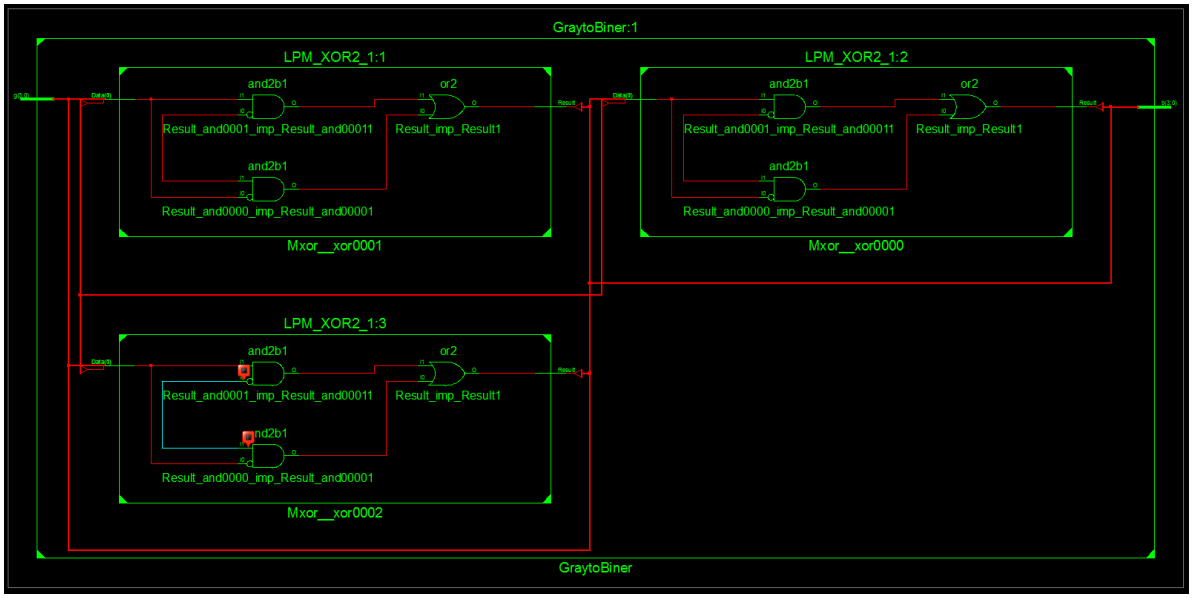



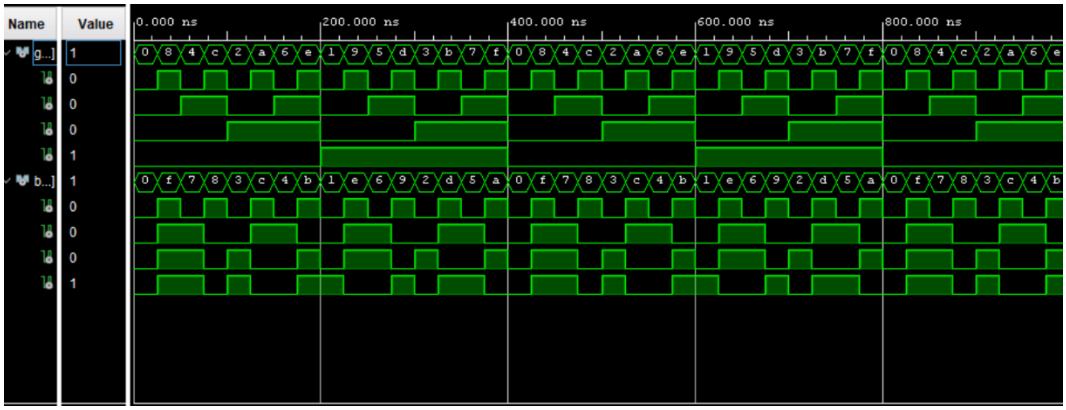



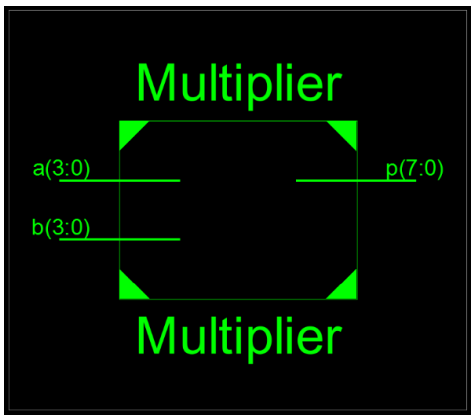
d. Gray to Binary

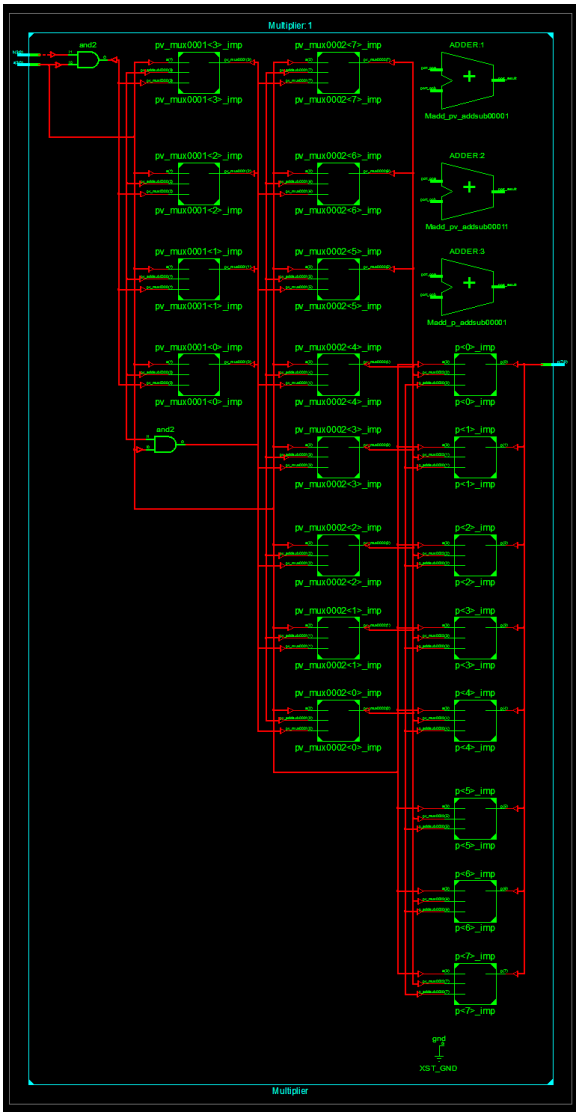


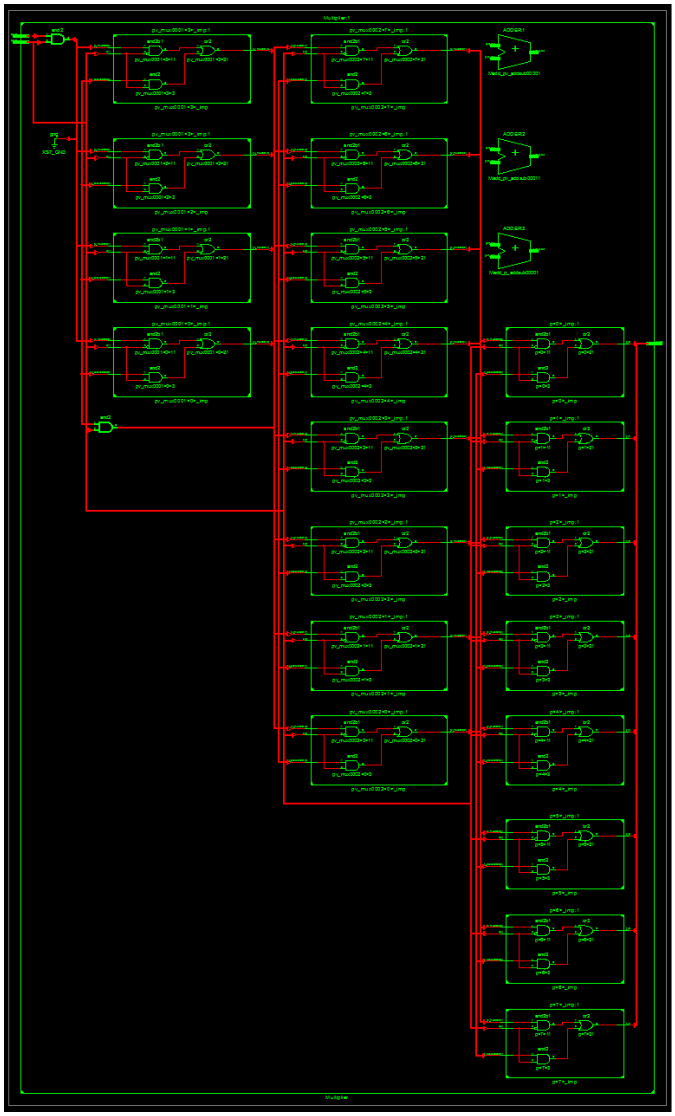



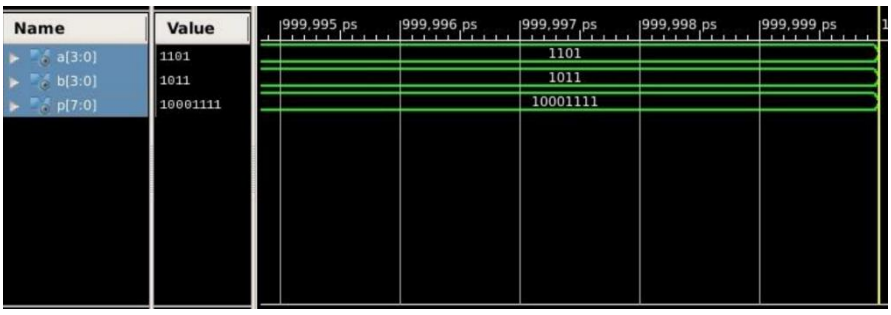



e. Multiplier

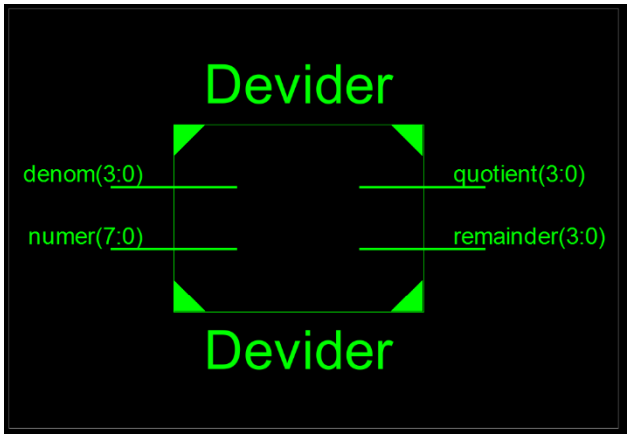


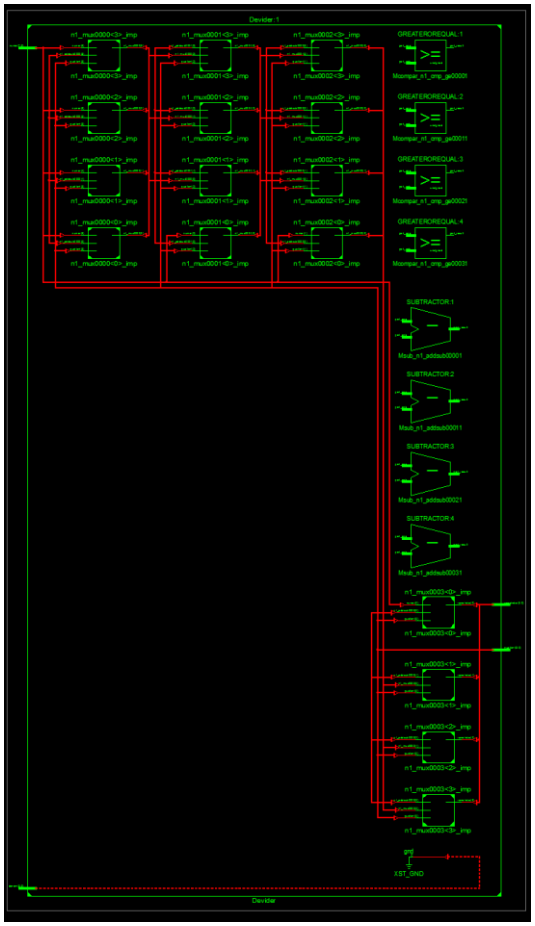



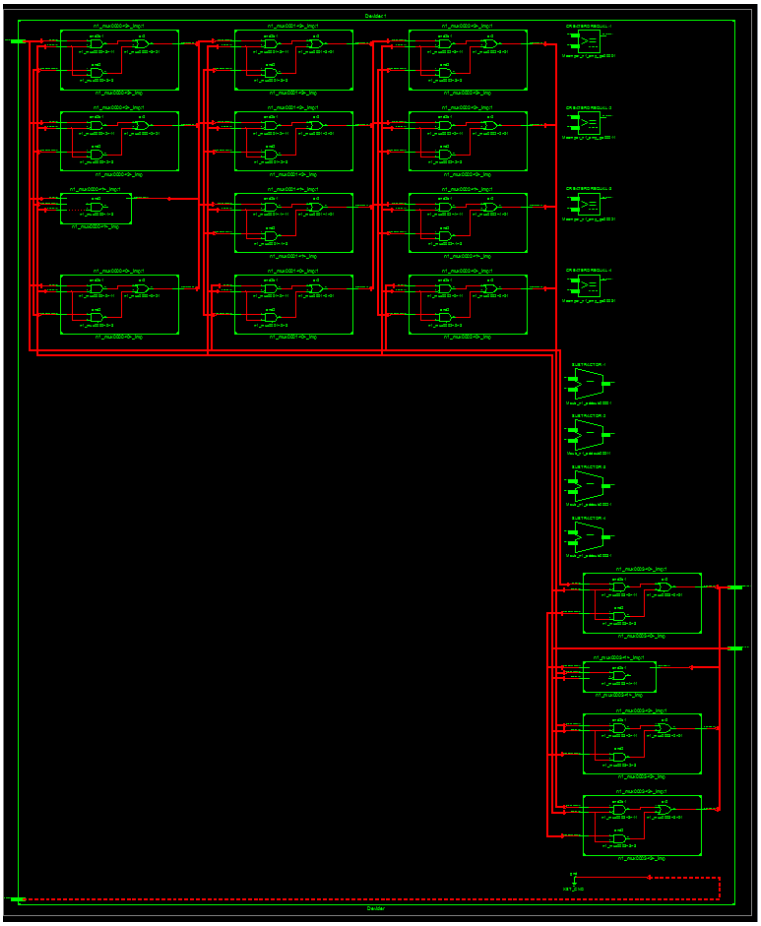


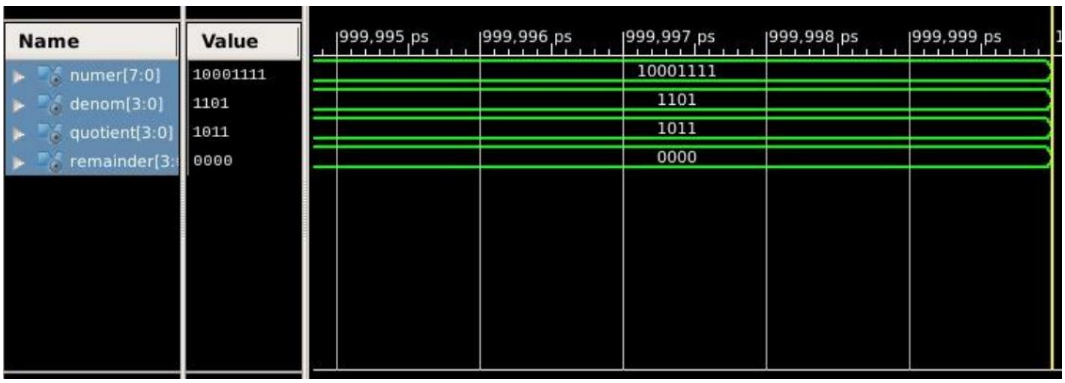


f. Divider

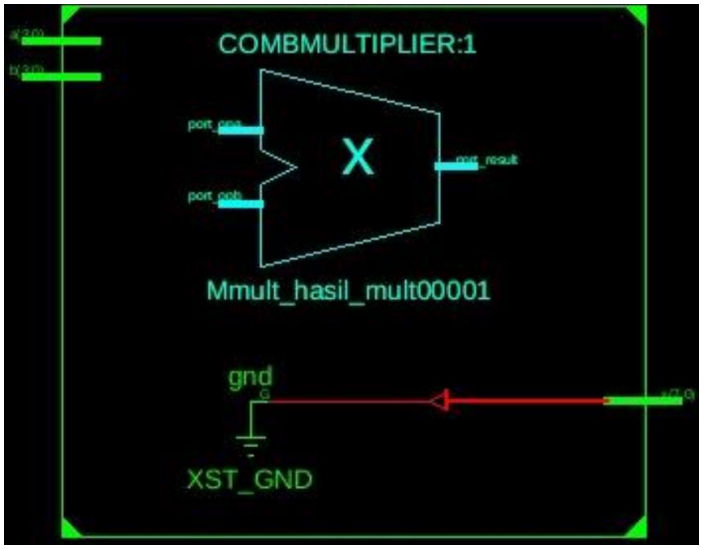

Devider
denom(3:0)
numer(7:0)
quotient(3:0)
remainder(3:0)
Devider

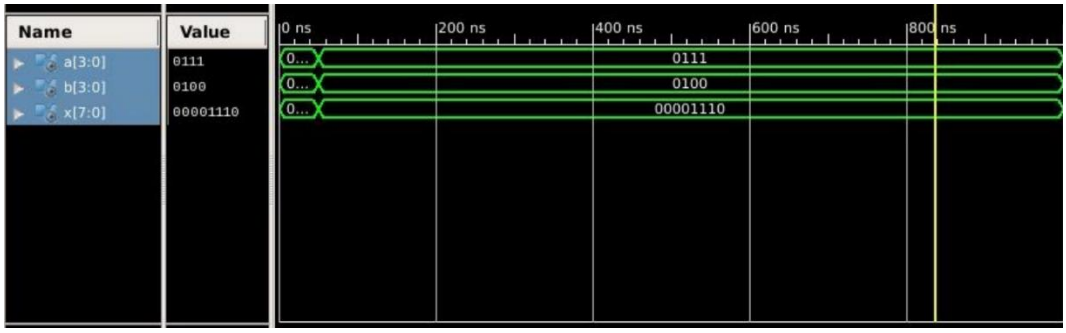





Name
Value
numer[7:0]
10001111
denom[3:0]
1101
quotient[3:0]
1011
remainder[3:
0000
999,995 ps
999,996 ps
999,997 ps
999,998 ps
999,999 ps
10001111
1101
1011
0000

g. Tugas


COMBMULTIPLIER:1
X
Mmult_hasil_mult00001
gnd
XST_GND


Name
Value
a[3:0]
0111
b[3:0]
0100
x[7:0]
00001110
0 ns
200 ns
400 ns
600 ns
800 ns
0111
0100
00001110

## 4. ANALISA

a) Decoder_3_to_8

Dalam implementasi VHDL, entitas decoder 3-ke-8 menggunakan pendekatan behavioral. Proses implementasinya melibatkan konversi vektor input menjadi integer dengan variabel conv_integer. Selanjutnya, terdapat iterasi dari 0 hingga 7 untuk menetapkan nilai output dengan kondisi if(i==j). Jika kondisi tersebut terpenuhi, nilai logika menjadi '1', jika tidak, nilai logika menjadi '0'. Proses ini menerjemahkan masukan 3-bit menjadi keluaran 8-bit sesuai dengan logika dekoder. Test bench dibuat untuk menguji fungsi dekoder ini, dimana sinyal input diinisialisasi dan stimulus dihasilkan untuk mengevaluasi keluaran.

b) Binary_to_BCD

Untuk konverter biner 8-bit ke BCD 10-bit, proses VHDL mengikuti pendekatan behavioral. Dalam proses ini, dideklarasikan sebuah proses yang menerima input biner 8-bit dan menghasilkan output BCD 10-bit. Pada proses tersebut, dideklarasikan sebuah variable z :STD_LOGIC_VECTOR(17 downto 0) yang berfungsi untuk menyimpan hasil konversi. Nilai-nilai z diinisialisasi dengan '0', kemudian nilai biner input ditempatkan di posisi yang sesuai dalam variabel z. Dilakukan iterasi untuk memastikan bahwa setiap digit BCD tidak melebihi nilai 9; jika melebihi, digit tersebut ditambah 3. Setelah itu, digit diperbarui dan nilai output p diatur sebagai nilai z dari bit ke-17 hingga ke-8. Test bench yang dibuat digunakan untuk menguji konverter ini dengan menginisialisasi sinyal input dan menimbulkan stimulus untuk memeriksa responsnya.

c) Binary_to_Gray

Implementasi VHDL untuk konverter kode biner ke Gray menggunakan arsitektur behavioral. Program VHDL ini bertujuan untuk mengonversi kode biner menjadi kode Gray. Entitasnya didefinisikan sebagai c_gray_code dengan input b dan output g, keduanya berupa vektor logika dengan panjang 4 bit. Prosesnya menggunakan operasi XOR untuk menghitung kode Gray untuk setiap bit dalam masukan biner. Output g dihasilkan dari proses ini. Test bench digunakan untuk menguji konverter ini dengan menyediakan stimulus pada input dan memeriksa keluaran yang dihasilkan.

d) Gray_to_binary

Program VHDL yang mengonversi kode Gray menjadi biner didefinisikan sebagai d_gray_code dengan input g dan output b, keduanya berupa vektor

logika dengan panjang 4 bit. Prosesnya menghitung nilai biner untuk setiap bit menggunakan operasi XOR. Output b dihasilkan dari proses ini.

e) Multiplier

Dengan menggunakan operator bitwise &, program VHDL mengimplementasikan pengganda 4-bit pada entitas multiplier dengan input a dan b sebagai vektor logika 4-bit, dan output p sebagai vektor logika 8-bit. Proses perkalian bit demi bit dilakukan dalam arsitektur behavioral. Setiap implementasi lainnya memiliki pendekatan yang serupa namun menggunakan operator * atau fungsi untuk menghitung hasil perkalian dari a dan b.

f) Divider

Implementasi pembagi dalam VHDL menggunakan arsitektur perilaku. Proses ini melakukan pembagian dengan menggunakan iterasi melalui proses, yang memproses pembilang dan penyebut untuk menghasilkan hasil bagi dan sisa. Untuk menguji pembagi ini, dibuatlah sebuah test bench yang menginisialisasi input dan menimbulkan stimulus untuk mengamati responsnya.

g) Tugas

Pada program tugas, VHDL mengimplementasikan operasi matematika x = (a×b)^2. Entitas ini memiliki dua input, yaitu vektor logika 4 bit a dan b, serta output x berupa vektor logika 8 bit. Di dalam bagian arsitekturnya yang berjenis Behavioral, terdapat sebuah fungsi bernama persamaan. Fungsi ini menerima dua parameter berupa bilangan bulat unsigned a dan b dengan panjang 4 bit, sesuai dengan tipe data yang disediakan oleh pustaka numeric_std. Tujuan dari fungsi ini adalah untuk menghasilkan nilai dari operasi matematika x = (a×b)^2. Dalam fungsi persamaan, terdapat variabel lokal bernama hasil yang dideklarasikan untuk menyimpan hasil dari operasi tersebut. Prosesnya dimulai dengan melakukan operasi perkalian antara a dan b, dan hasilnya kemudian dibagi dua menggunakan operator /. Setelah itu, nilai hasil dari operasi tersebut dikembalikan sebagai nilai dari fungsi. Pada bagian utama arsitekturnya, dua sinyal bernama nilai1 dan nilai2 dideklarasikan sebagai bilangan bulat unsigned dengan panjang 4 bit. Sinyal-sinyal ini diinisialisasi dengan nilai dari input a dan b. Selanjutnya, fungsi persamaan dipanggil dengan parameter nilai1 dan nilai2, dan hasilnya dikonversi ke tipe data std_logic_vector (bilangan biner) sebelum disimpan di output x.

## 5. KESIMPULAN

Kesimpulan dari perbandingan penggunaan for loop statement dalam VHDL adalah implementasi VHDL dalam berbagai fungsi seperti decoder, konverter, pengganda, pembagi, dan operasi matematika mengikuti pendekatan behavioral atau perilaku dalam arsitekturnya. Penggunaan loop for statement sangat relevan dalam konteks ini karena loop for statement memungkinkan pengulangan perintah-perintah yang sama secara otomatis sesuai dengan jumlah iterasi yang diinginkan.

Pada entitas decoder 3-ke-8, loop for statement mungkin digunakan dalam proses iterasi untuk menetapkan nilai output dengan kondisi if(i==j). Hasilnya adalah penerjemahan masukan 3-bit menjadi keluaran 8-bit sesuai dengan logika dekoder. Uji coba dilakukan dengan test bench yang menginisialisasi sinyal input dan menimbulkan stimulus untuk mengevaluasi responsnya, di mana loop for statement juga dapat digunakan untuk mengatur iterasi dalam proses uji coba.

Konverter biner 8-bit ke BCD 10-bit juga mengikuti pendekatan behavioral, di mana nilai-nilai z diinisialisasi dengan '0' dan digit BCD diperbarui jika melebihi nilai 9. Loop for statement dapat digunakan dalam proses iterasi untuk memastikan setiap digit BCD tidak melebihi nilai 9.

Implementasi konverter kode biner ke Gray dan sebaliknya menggunakan arsitektur behavioral dengan penggunaan operasi XOR untuk menghitung kode Gray atau biner. Loop for statement dapat digunakan dalam proses iterasi untuk menghitung kode Gray atau biner untuk setiap bit dalam masukan biner.

Pada entitas pengganda 4-bit, loop for statement mungkin digunakan dalam proses iterasi untuk perkalian bit demi bit yang dilakukan dalam arsitektur behavioral. Sementara pada pembagi, loop for statement dapat digunakan dalam iterasi melalui proses pembagian dan penghitungan hasil bagi serta sisa. Sehingga loop for statement merupakan elemen kunci dalam implementasi fungsi-fungsi VHDL yang menggunakan pendekatan behavioral. Loop for statement memungkinkan pengulangan perintah-perintah yang diperlukan secara otomatis sesuai dengan kebutuhan desain, sehingga sangat relevan dalam proses pengembangan dan uji coba rangkaian-rangkaian VHDL.